

LAB ASSIGNMENT 1: DOCKER INSTALLATION, DOCKER HUB REGISTRATION, AND NGINX CONTAINER DEPLOYMENT

Objective

The objective of this lab is to:

- Install Docker Desktop successfully.
- Register a Docker Hub account using a college email ID.
- Verify Docker installation using Docker commands.
- Pull and run an Nginx container using Docker.
- Understand the complete Docker container lifecycle workflow.

Theory

- **Docker:** Docker is an open-source containerization platform that enables developers to package applications and their dependencies into lightweight containers. Containers ensure applications run consistently across different systems and environments.
- **Docker Hub:** Docker Hub is a cloud-based container registry service used to store, manage, and distribute Docker images. Developers can push their custom images to Docker Hub and pull images from anywhere.
- **Nginx Container:** Nginx is a high-performance web server commonly used for hosting websites and reverse proxy services. Docker allows Nginx to run inside isolated containers without requiring manual server configuration.
- **Containerization:** Containerization is a virtualization technique where applications run in isolated user spaces called containers. It improves portability, scalability, and deployment consistency.

Part 1: Download and Install Docker Desktop

Step 1: Visit Docker Official Website

- Open a web browser and visit the official Docker website: <https://www.docker.com/>
- Download Docker Desktop according to your operating system:
 - Windows 10/11 (64-bit)
 - macOS (Intel or Apple Silicon)

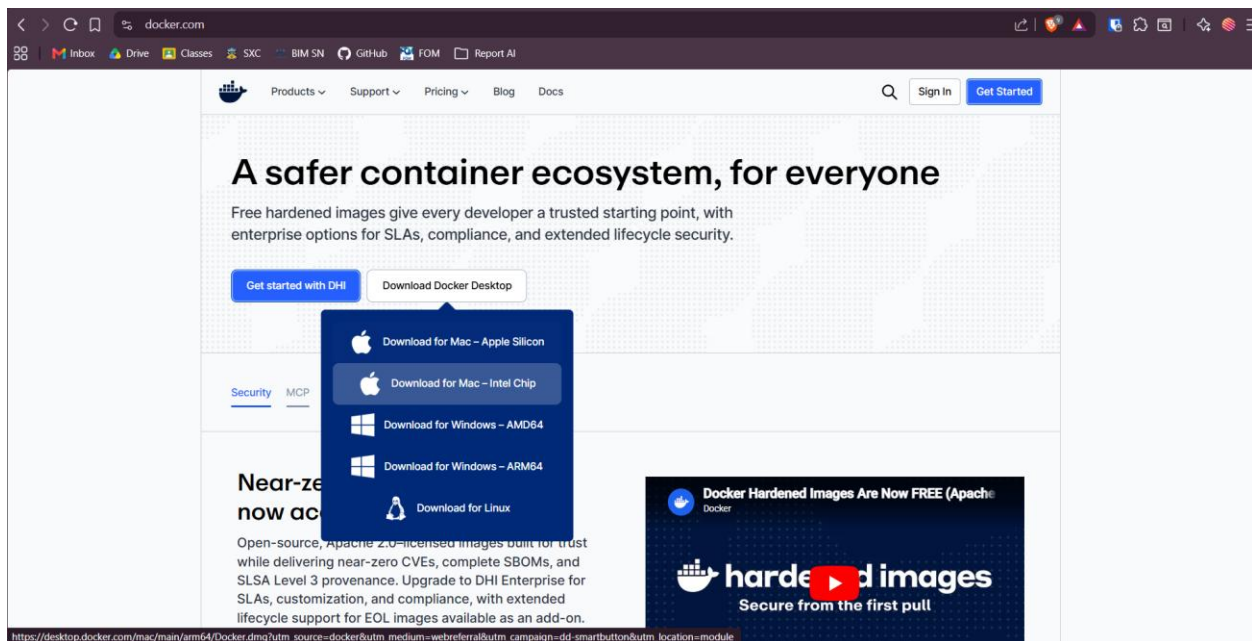


Figure 1.1 Docker official website

Step 2: Install Docker Desktop

- Run the installer and follow the installation process.
- For Windows:
 - Enable WSL 2 if prompted.
 - Complete all required configurations.
- Launch Docker Desktop after installation.

Step 3: Launch Docker Desktop

After installation, open Docker Desktop and verify that it launches successfully.

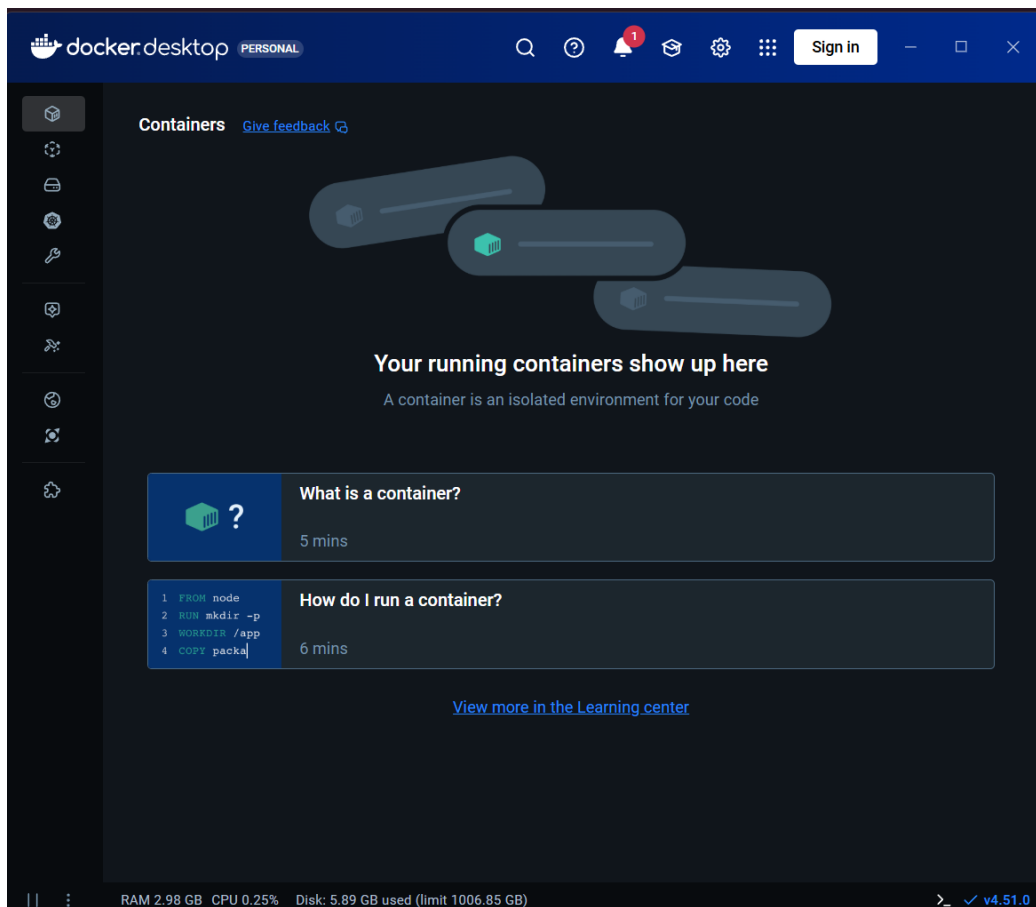


Figure 1.2 Docker desktop app in windows

Part 2: Register Docker Account

Step 1: Open Docker Hub

- Visit Docker Hub from the official Docker website.
- Click on the “Sign Up” button.
- Register using your official college email ID.

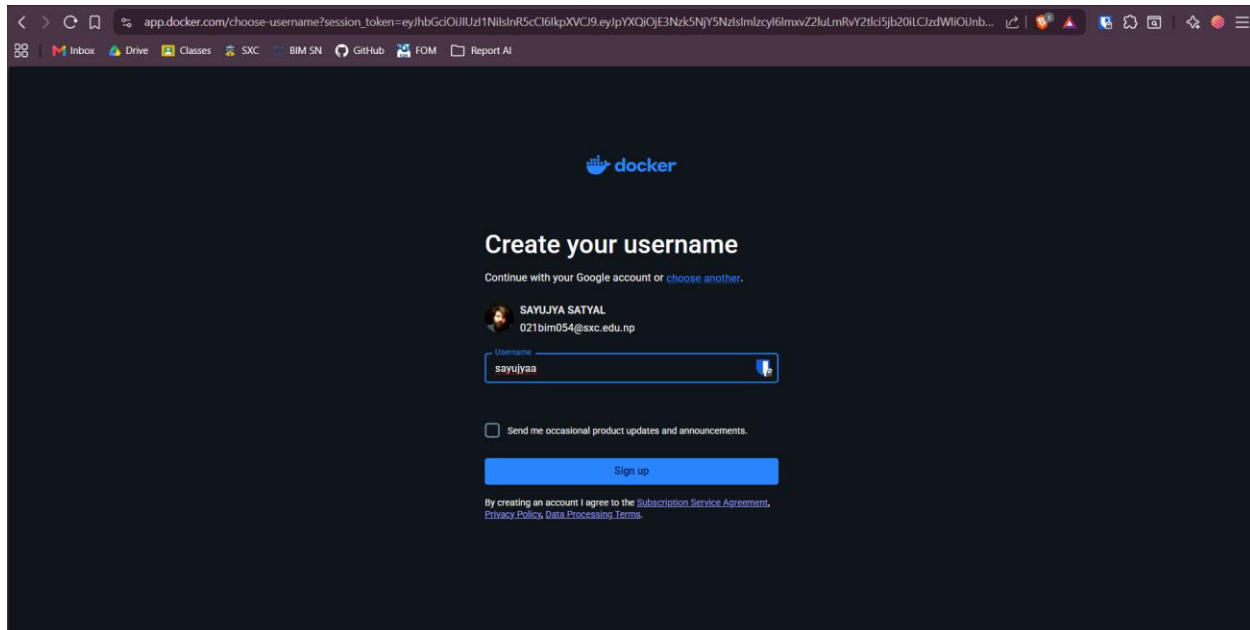


Figure 1.3 Signing in into docker hub

Step 2: Verify Email Address

Open your email inbox and verify your Docker account through the verification email.

Screenshot

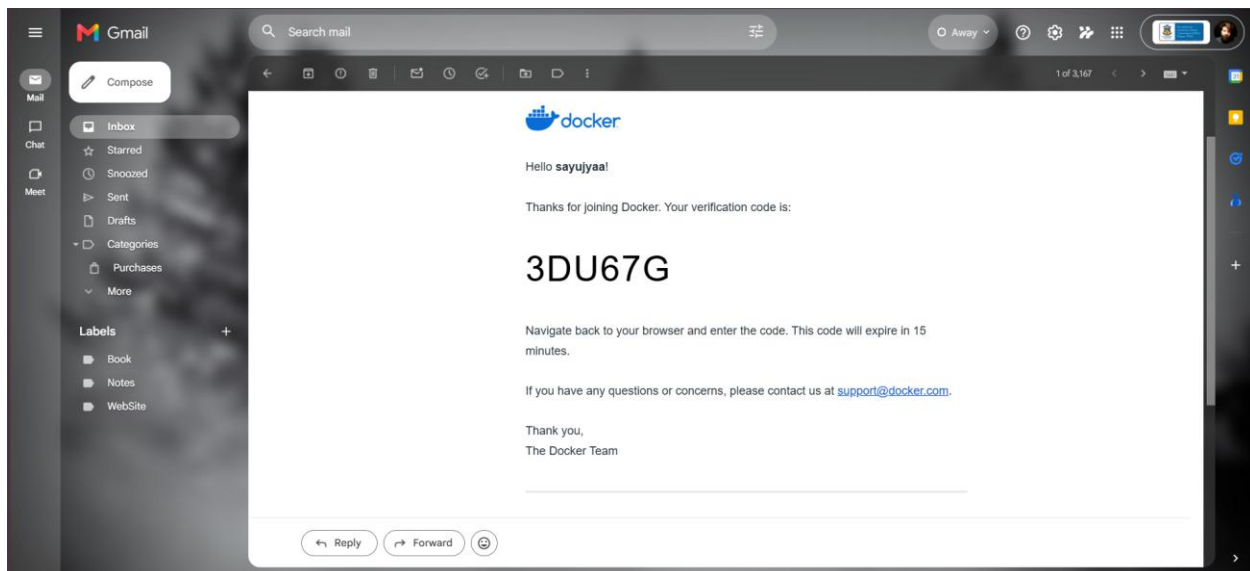


Figure 1.4 Docker otp during signup

Step 3: Login to Docker Hub

Log in using your Docker Hub credentials.

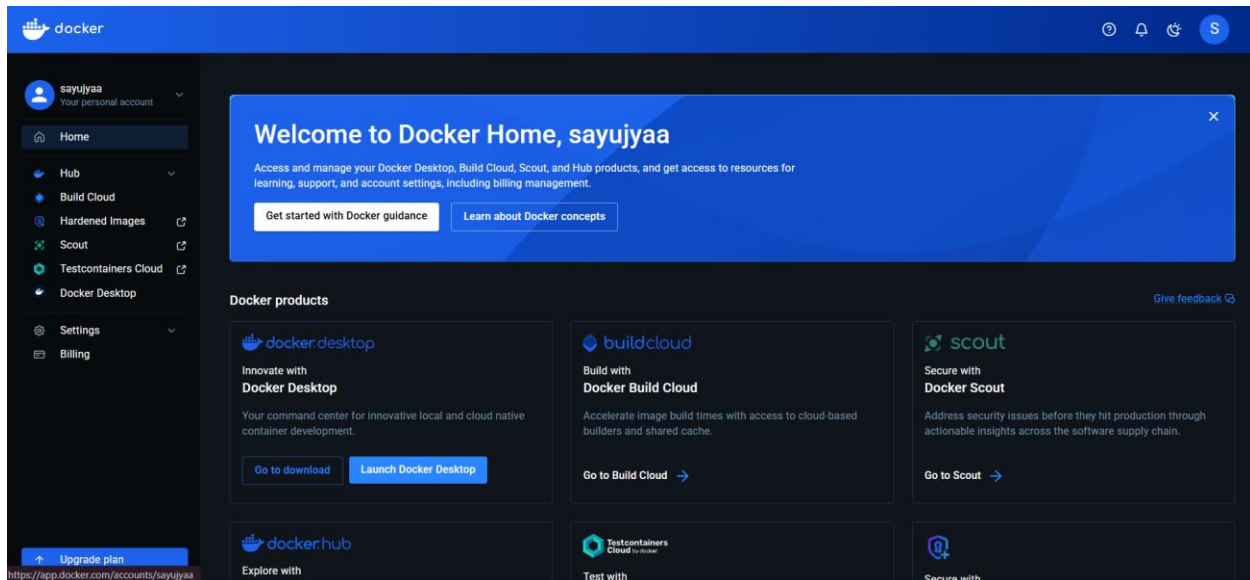


Figure 1.5 Successful signup in docker hub

Part 3: Verify Docker Installation

- Open Command Prompt
- Run the following command: `docker --version`
- This command verifies that Docker has been installed successfully.

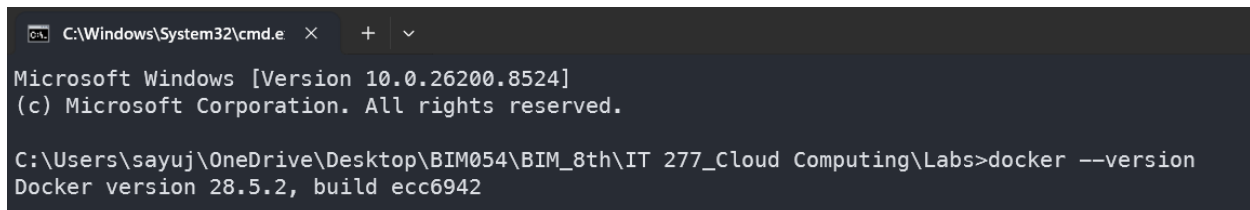


Figure 1.6 Docker installation verification

Part 4: Pull and Run Nginx Container

Step 1: Pull Nginx Image

- Run the following command: `docker pull nginx`
- Docker downloads the official Nginx image from Docker Hub.

```
C:\Users\sayuj\OneDrive\Desktop\BIM054\BIM_8th\IT 277_Cloud Computing\Labs>docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
7f8b1a2b17d8: Pull complete
13fd728be9eb: Pull complete
5431d0092ffd: Pull complete
830625e1ac85: Pull complete
b4a248c845e5: Pull complete
5b4d6ff92fc4: Pull complete
45381ecb0e2f: Pull complete
Digest: sha256:5aca99593157f4ae539a5dec1092a0ad8762f8e2eb1789085a13a0f5622369f6
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

Figure 1.7 Pulling in the nginx image

Step 2: Create and Run Nginx Container

- Run the following command: `docker run -d -p 8080:80 --name sayujya-nginx nginx`
- Explanation:
 - `-d` runs the container in detached mode.
 - `-p 8080:80` maps local port 8080 to container port 80.
 - `--name` assigns a custom container name.

```
C:\Users\sayuj\OneDrive\Desktop\BIM054\BIM_8th\IT 277_Cloud Computing\Labs>docker run -d -p 8080:80 --name sayujya-nginx nginx
929376034d1b679181be1cc1726d6552a99d2a09895564a2bea05f9445d5fefcd
C:\Users\sayuj\OneDrive\Desktop\BIM054\BIM_8th\IT 277_Cloud Computing\Labs>
```

Figure 1.8 Create and run nginx container

Step 3: Verify Running Container

- Run the following command: `docker ps`
- This displays all currently running containers.

```
C:\Users\sayuj\OneDrive\Desktop\BIM054\BIM_8th\IT 277_Cloud Computing\Labs>docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
929376034d1b   nginx     "/docker-entrypoint..." 2 minutes ago  Up 2 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  sayujya-nginx
C:\Users\sayuj\OneDrive\Desktop\BIM054\BIM_8th\IT 277_Cloud Computing\Labs>_
```

Figure 1.9 Verify the container is running

Step 4: Test Nginx in Browser

- Open a web browser and visit: `http://localhost:8080`
- The Nginx welcome page should appear successfully.

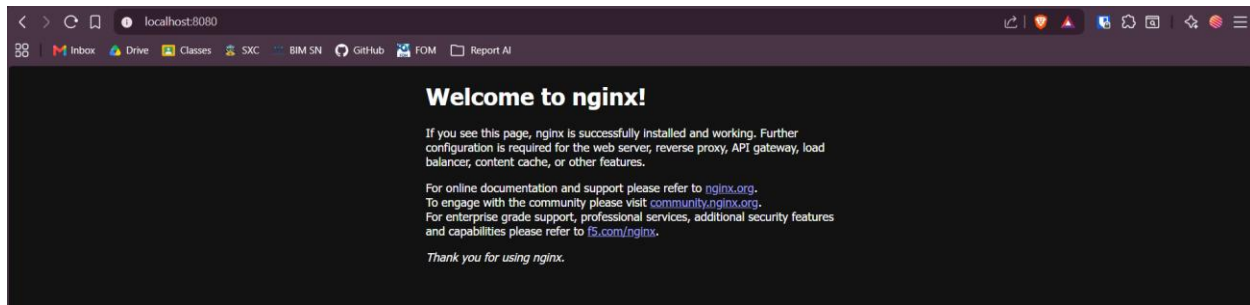


Figure 1.10 Nginx run test in browser

Step 5: Verify in Docker Desktop Dashboard

Open Docker Desktop and verify that the Nginx container is running.

Screenshot

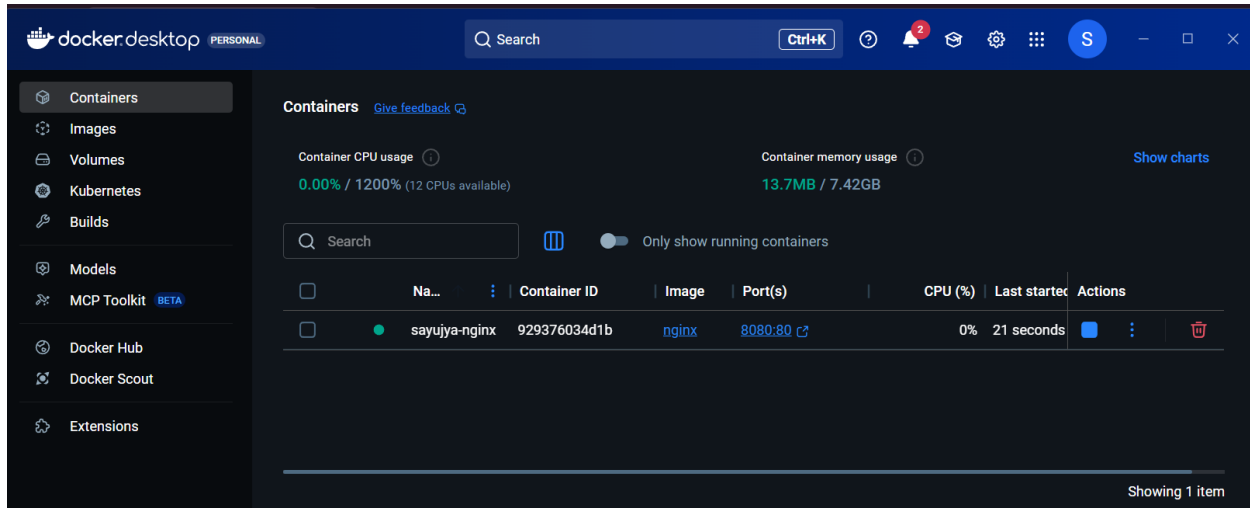


Figure 1.11 Final verification in docker desktop dashboard

Result

Docker Desktop was installed successfully. A Docker Hub account was created and verified using the college email ID. The Nginx Docker container was successfully pulled, executed, and accessed through the browser using localhost port mapping.

Conclusion

This lab provided practical knowledge about Docker installation, Docker Hub registration, Docker commands, container execution, and web server deployment using Nginx containers. The

experiment demonstrated the basic workflow used in modern containerized application deployment.

Docker Complete Lifecycle Summary

1. Install Docker

Docker is first installed on the system to enable containerization.

Examples:

- Docker Desktop for Windows/macOS
- Docker Engine for Linux

Purpose:

To provide a platform for building and running containers.

2. Verify Installation

Commands used:

```
docker --version
```

Purpose:

To verify that Docker is properly installed and running.

3. Create Project

Create a project directory for application development.

Example:

```
mkdir myapp  
cd myapp
```

The project may contain:

- Source code
- Dependencies
- Configuration files

4. Create Dockerfile

Create a file named Dockerfile.

Example:

```
FROM node:18
```



```
WORKDIR /app
COPY . .
RUN npm install
CMD ["node", "app.js"]
```

Purpose:

Defines how Docker builds the application image.

5. Build Docker Image

Command:

```
docker build -t myapp:v1 .
```

Explanation:

- -t specifies image tag
- myapp:v1 is image name and version

Check images using:

```
docker images
```

6. Run Container

Command:

```
docker run -d -p 3000:3000 --name mycontainer myapp:v1
```

Purpose:

Runs the application inside a container.

7. Test and View Logs

Test application in browser:

```
http://localhost:3000
```

View logs:

```
docker logs mycontainer
```

Purpose:

Used for debugging and monitoring.

8. Stop and Remove Container

Stop container:

```
docker stop mycontainer
```

Remove container:

```
docker rm mycontainer
```

Remove image:

```
docker rmi myapp:v1
```

9. Login to Docker Hub**Command:**

```
docker login
```

Purpose:

Authenticate with Docker Hub registry.

10. Tag Docker Image**Command:**

```
docker tag myapp:v1 username/myapp:v1
```

Purpose:

Prepares the image for pushing to Docker Hub.

11. Push Docker Image**Command:**

```
docker push username/myapp:v1
```

Purpose:

Uploads image to Docker Hub.

12. Pull Image on Server**Command:**

```
docker pull username/myapp:v1
```

Purpose:

Downloads the image to a production server.

13. Deploy Application**Command:**

```
docker run -d -p 80:3000 username/myapp:v1
```

Purpose:

Deploys the application to production.

14. Rebuild New Version**Command:**

```
docker build -t myapp:v2 .
```

Purpose:

Creates a new version after code updates.

15. Push Updated Version**Commands:**

```
docker tag myapp:v2 username/myapp:v2
```

```
docker push username/myapp:v2
```

Purpose:

Uploads updated image version.

16. Redeploy Updated Application**Commands:**

```
docker pull username/myapp:v2
```

```
docker stop oldcontainer
```

```
docker rm oldcontainer
```

```
docker run -d -p 80:3000 username/myapp:v2
```

Purpose:

Updates the running production application.

Docker Lifecycle Flow

- Development Machine: Code → Dockerfile → Build → Run → Test
- Docker Registry: Tag → Push
- Production Server: Pull → Run → Deploy
- Update Cycle: Modify Code → Rebuild → Push → Pull → Redeploy

Final Summary

Docker lifecycle follows a continuous workflow:

Build → Run → Test → Push → Deploy → Update → Repeat

Benefits include:

- Consistent environments
- Faster deployment
- Better scalability
- Simplified version control
- Easier application management

LAB ASSIGNMENT 2: DOCKERIZE A REACT PROJECT FROM GITHUB

Objective

The objective of this lab is to:

- Clone an existing React project from GitHub.
- Create a Dockerfile for the React application.
- Build a Docker image using Docker.
- Run the Docker container locally.
- Push the Docker image to Docker Hub.
- Learn the workflow of deploying React applications using Docker containers.

Theory

- **React Application:** React is a JavaScript library used for building interactive user interfaces and single-page web applications. React applications are commonly deployed using web servers such as Nginx or Apache.
- **Docker Image:** A Docker image is a lightweight, executable package containing application code, dependencies, libraries, and configurations required to run an application.
- **Docker Container:** A Docker container is a running instance of a Docker image. Containers provide isolated environments for applications.
- **Nginx Web Server:** Nginx is a high-performance web server used to serve static files and host frontend applications such as React production builds.
- **Docker Hub:** Docker Hub is a cloud-based registry used to store and distribute Docker images.

Requirements

- Docker Desktop installed
- Git installed
- Docker Hub account
- Existing React project hosted on GitHub

Part 1: Clone React Project from GitHub

Step 1: Open Terminal or Command Prompt

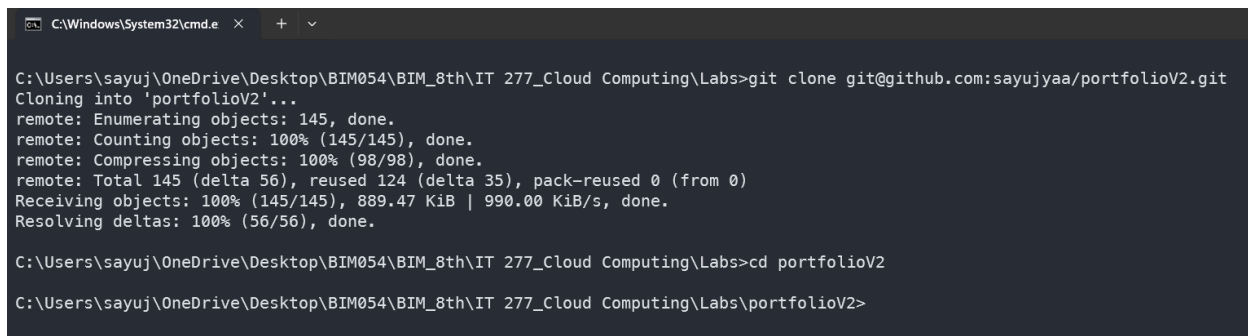
Open Terminal, PowerShell, or Command Prompt on your system.

Step 2: Clone the GitHub Repository

- Run the following command: `git clone git@github.com:sayujyaa/portfolioV2.git`
- Move into the project directory: `cd portfolioV2`

Explanation:

- `'git clone'` downloads the React project from GitHub.
- `'cd'` changes the current directory to the project folder.



```
C:\Windows\System32\cmd.exe
C:\Users\sayuj\OneDrive\Desktop\BIM054\BIM_8th\IT 277_Cloud Computing\Labs>git clone git@github.com:sayujyaa/portfolioV2.git
Cloning into 'portfolioV2'...
remote: Enumerating objects: 145, done.
remote: Counting objects: 100% (145/145), done.
remote: Compressing objects: 100% (98/98), done.
remote: Total 145 (delta 56), reused 124 (delta 35), pack-reused 0 (from 0)
Receiving objects: 100% (145/145), 889.47 KiB | 990.00 KiB/s, done.
Resolving deltas: 100% (56/56), done.

C:\Users\sayuj\OneDrive\Desktop\BIM054\BIM_8th\IT 277_Cloud Computing\Labs>cd portfolioV2

C:\Users\sayuj\OneDrive\Desktop\BIM054\BIM_8th\IT 277_Cloud Computing\Labs\portfolioV2>
```

Figure 2.1 Project clone and move

Part 2: Create Dockerfile

Step 1: Create Dockerfile

Inside the React project directory, create a file named: Dockerfile

Step 2: Add Dockerfile Content

Example Dockerfile using Nginx:

```
FROM node:18 AS build
WORKDIR /app
COPY . .
RUN npm install
RUN npm run build
FROM nginx:latest
```

```
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Explanation:

- The first stage builds the React application.
- The second stage uses Nginx to serve the production build.

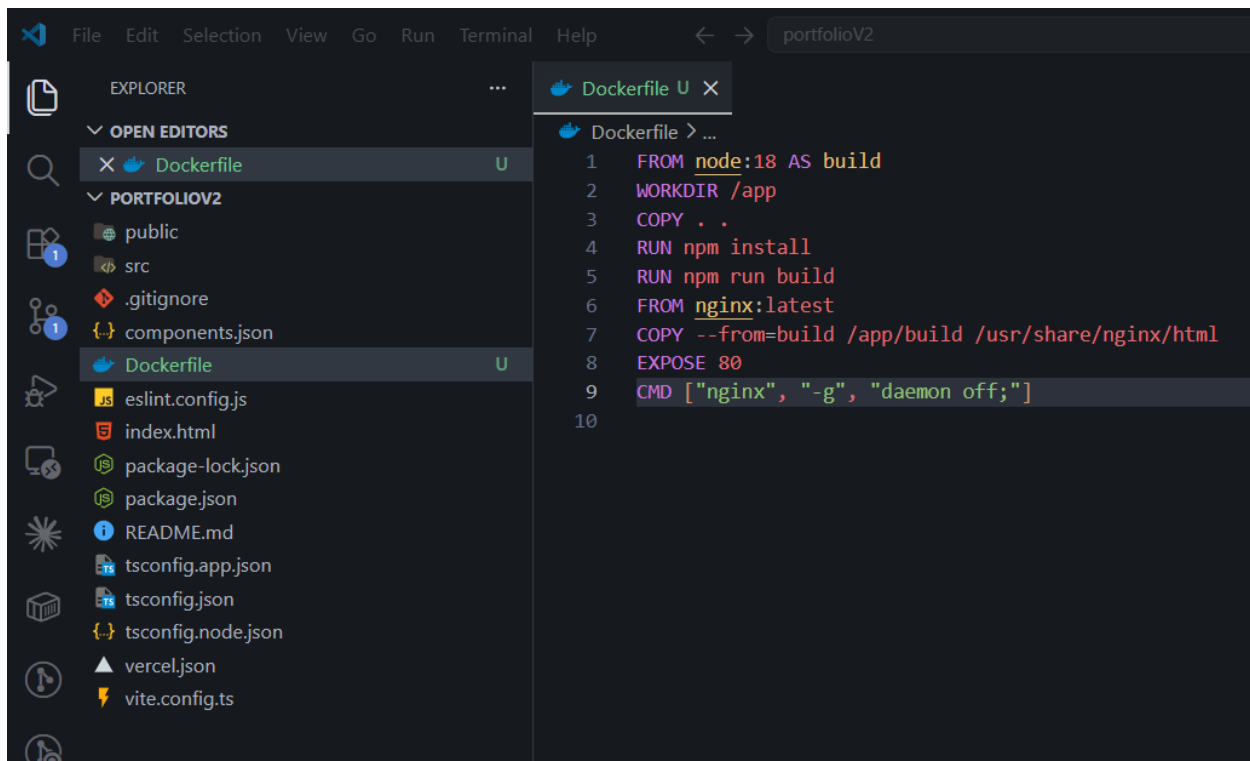


Figure 2.2 Create and add content to the docker file

Part 3: Build Docker Image

Step 1: Build Docker Image

Run the following command:

```
docker build -t sayujya-react .
```

Explanation:

- `docker build` creates the Docker image.

- `-t` assigns a tag or image name.
- `..` indicates the current project directory.

Step 2: Verify Image

Run:

```
docker images
```

This displays the list of available Docker images.

```

■ @ sayuj on ~/OneDrive/Desktop/BIM054/BIM_8th/IT 277_Cloud Computing/Labs/portfolioV2
● # docker build -t sayujya-react .
[+] Building 20.4s (14/14) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 239B
=> [internal] load metadata for docker.io/library/nginx:latest
=> [internal] load metadata for docker.io/library/node:18
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [build 1/5] FROM docker.io/library/node:18@sha256:c6ae79e38498325db67193d391e6ec1d224d96c693a8a4d943498556716d3783
=> => resolve docker.io/library/node:18@sha256:c6ae79e38498325db67193d391e6ec1d224d96c693a8a4d943498556716d3783
=> CACHED [stage-1 1/2] FROM docker.io/library/nginx:latest@sha256:5aca99593157f4ae539a5dec1092a0ad8762f8e2eb1789085a13a0f5622369f6
=> => resolve docker.io/library/nginx:latest@sha256:5aca99593157f4ae539a5dec1092a0ad8762f8e2eb1789085a13a0f5622369f6
=> [internal] load build context
=> => transferring context: 863.82kB
=> CACHED [build 2/5] WORKDIR /app
=> [build 3/5] COPY . .
=> [build 4/5] RUN npm install
=> [build 5/5] RUN npm run build
=> [stage-1 2/2] COPY --from=build /app/dist /usr/share/nginx/html
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:99d3a62c23e27c28b7327234642070b1d27e6d52bae77d1c0aaf66377333b205
=> => exporting config sha256:831018adcd92ba64d30543b6e1551ee9d552498b963f4d9400f57baeb757556d
=> => exporting attestation manifest sha256:e6992d8eefcb65d01a2635af8ec0b6d2ca2f348a923bf810ece828b9ad80aaad
=> => exporting manifest list sha256:f6229e72976deba08fbbfb560b666f5637b0788cb9c14972aeadd0fa72dbed764
=> => naming to docker.io/library/sayujya-react:latest
=> => unpacking to docker.io/library/sayujya-react:latest

■ @ sayuj on ~/OneDrive/Desktop/BIM054/BIM_8th/IT 277_Cloud Computing/Labs/portfolioV2
● # docker images
REPOSITORY          TAG             IMAGE ID        CREATED         SIZE
sayujya-react       latest         f6229e72976d   About a minute ago   240MB
nginx               latest        5aca99593157   5 days ago         238MB

```

Figure 2.3 Add Dockerfile and create docker image

Part 4: Run Docker Container

Step 1: Run Container

Run the following command:

```
docker run -d -p 8081:80 sayujya-react
```

Explanation:

- `-d` runs the container in detached mode.
- `-p 8080:80` maps local port 8080 to container port 80.

Step 2: Verify Running Container

Run:

`docker ps`

This shows all running containers.

```

sayuj on ~/OneDrive/Desktop/BIM054/BIM_8th/IT 277_Cloud Computing/Labs/portfolioV2 main
# docker run -d -p 8081:80 sayujya-react
0695ff72a69327c48418d96004520185731dc25eadc9d96f01c776ca6f2737ef

sayuj on ~/OneDrive/Desktop/BIM054/BIM_8th/IT 277_Cloud Computing/Labs/portfolioV2 main
# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
0695ff72a693   sayujya-react  "/docker-entrypoint.    23 seconds ago Up 9 seconds  0.0.0.0:8081->80/tcp, [::]:8081->80/tcp   gifted_boyd
929376034d1b   nginx          "/docker-entrypoint.    57 minutes ago Up 53 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   sayujya-nginx

```

Figure 2.4 Running the docker container

Part 5: Test React Application

Open a web browser and visit: `http://localhost:8081`

The React application should load successfully.

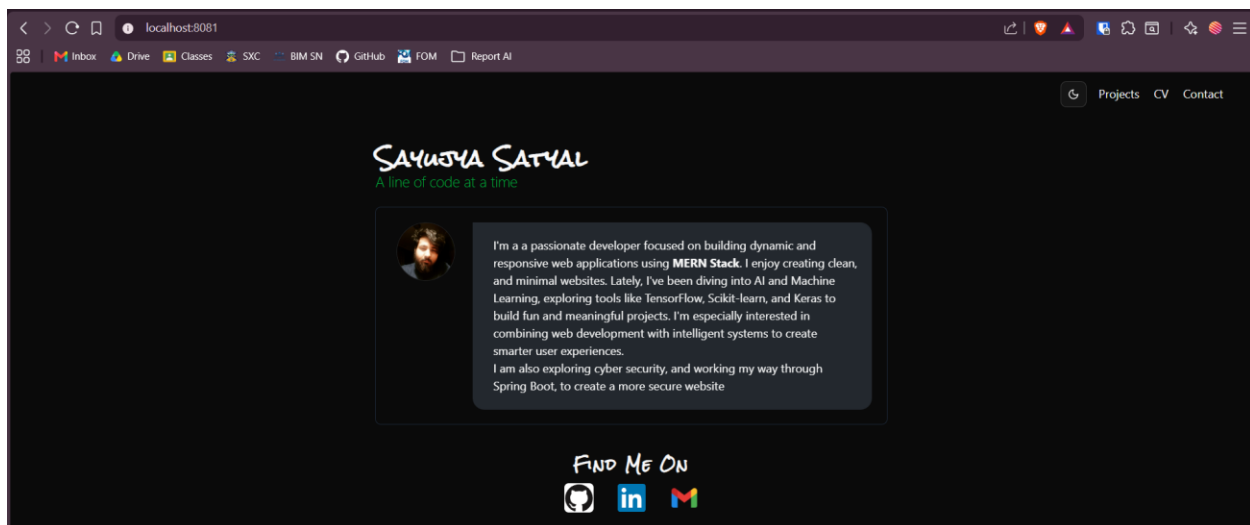


Figure 2.5 React application hosting successfully

Part 6: Push Docker Image to Docker Hub

Step 1: Login to Docker Hub

Run: `docker login` and ensure you are logged in inside the docker desktop app.

```

■ @ sayuj on ■ ~/OneDrive/Desktop/BIM054/BIM_8th/IT 277_Cloud Computing/Labs/portfolioV2 🐚main
● # docker login
Authenticating with existing credentials... [Username: sayujyaa]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded

```

Figure 2.6 Successful Login with Docker

Step 2: Tag Docker Image

Run: `docker tag sayujya-react sayujyaa/sayujya-react:latest`

Explanation:

- Tags the local image for Docker Hub upload.

```

■ @ sayuj on ■ ~/OneDrive/Desktop/BIM054/BIM_8th/IT 277_Cloud Computing/Labs/portfolioV2 🐚main
● # docker tag sayujya-react sayujyaa/sayujya-react:latest

```

Figure 2.7 Successful tag of docker image

Step 3: Push Docker Image

Run: `docker push sayujyaa/sayujya-react:latest`

Explanation:

- Uploads the Docker image to Docker Hub registry.

```

■ @ sayuj on ■ ~/OneDrive/Desktop/BIM054/BIM_8th/IT 277_Cloud Computing/Labs/portfolioV2 🐚main
● # docker push sayujyaa/sayujya-react:latest
The push refers to repository [docker.io/sayujyaa/sayujya-react]
7f8b1a2b17d8: Mounted from library/nginx
5b4d6ff92fc4: Mounted from library/nginx
830625e1ac85: Mounted from library/nginx
45381ecb0e2f: Mounted from library/nginx
13fd728be9eb: Mounted from library/nginx
f5347872a237: Pushed
c28aff39e96d: Pushed
5431d0092ffd: Mounted from library/nginx
b4a248c845e5: Mounted from library/nginx
latest: digest: sha256:f6229e72976deba08fbbfb560b666f5637b0788cb9c14972aead0fa72dbed764 size: 856

```

Figure 2.8 Successful push of docker image

Result

The React project was successfully dockerized using Docker and Nginx. The Docker image was built, executed locally, and pushed successfully to Docker Hub.

Conclusion

This lab provided practical experience in containerizing a React application using Docker. The experiment demonstrated how Docker simplifies application deployment, portability, and distribution using Docker Hub.

LAB ASSIGNMENT 3: ORACLE VM VIRTUALBOX INSTALLATION WITH UBUNTU SERVER 26.04 AND AAPANEL DEPLOYMENT

Objective

The objective of this lab is to:

- Install Oracle VM VirtualBox.
- Install Ubuntu Server 26.04 without GUI.
- Configure NAT networking and static IP address.
- Enable SSH access from VS Code terminal.
- Install aaPanel and Nginx web server.
- Configure firewall rules for required ports.
- Access and test the aaPanel dashboard through the browser.

Theory

- **Oracle VM VirtualBox:** Oracle VM VirtualBox is a virtualization software used to create and manage virtual machines. It allows multiple operating systems to run on a single physical computer.
- **Ubuntu Server:** Ubuntu Server is a Linux-based operating system designed for servers and cloud environments. Unlike Ubuntu Desktop, it does not include a graphical interface and is optimized for command-line administration.
- **NAT Networking:** NAT (Network Address Translation) networking allows the virtual machine to access external networks using the host system's internet connection while remaining isolated from the external network.
- **SSH (Secure Shell):** SSH is a secure network protocol used for remote access and command-line management of Linux servers.
- **aaPanel:** aaPanel is a lightweight hosting control panel used for managing websites, databases, FTP services, and web servers such as Nginx and Apache.

Requirements

- Oracle VM VirtualBox
- Ubuntu Server 26.04 ISO
- Stable internet connection
- VS Code with SSH extension
- Minimum 4 GB RAM
- Minimum 25 GB Storage

Part 1: Install Oracle VM VirtualBox

- Visit the official VirtualBox website: <https://www.virtualbox.org/>
- Download the installer for your operating system.
- Run the installer and complete the installation process using default settings.

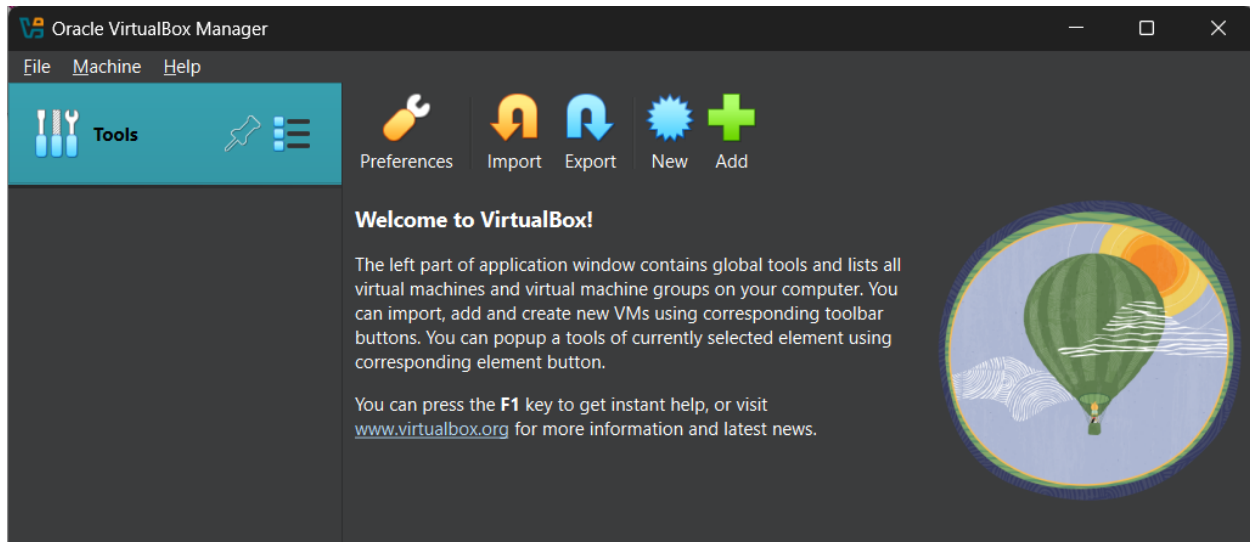


Figure 3.1 Virtualbox after successful installation

Part 2: Install Ubuntu Server 26.04

Step 1: Download the Ubuntu Server ISO file

- Visit the official Ubuntu Server website: <https://ubuntu.com/download/server>
- Download Ubuntu Server 26.04 ISO image.

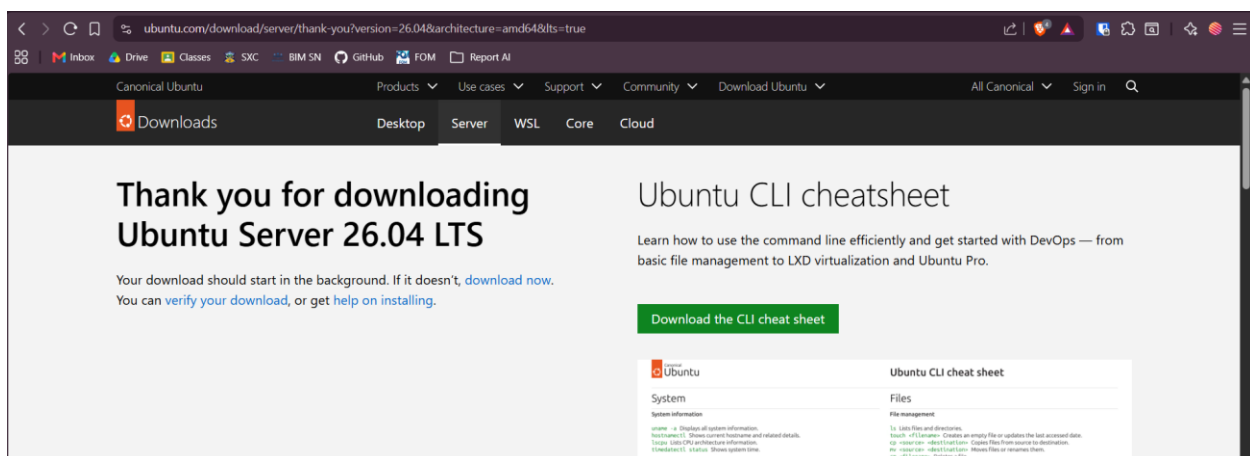


Figure 3.2 Downloading the ubuntu server 26.04

Step 2: Create New Virtual Machine

- Open VirtualBox and click on “New”.
- **Configure:**
 - Name: Ubuntu Server
 - Type: Linux
 - Version: Ubuntu (64-bit)
- **Allocate:**
 - RAM: 4096 MB
 - Processors: 6 CPUs
 - Storage: 25 GB dynamically allocated
- Attach the Ubuntu Server ISO file.

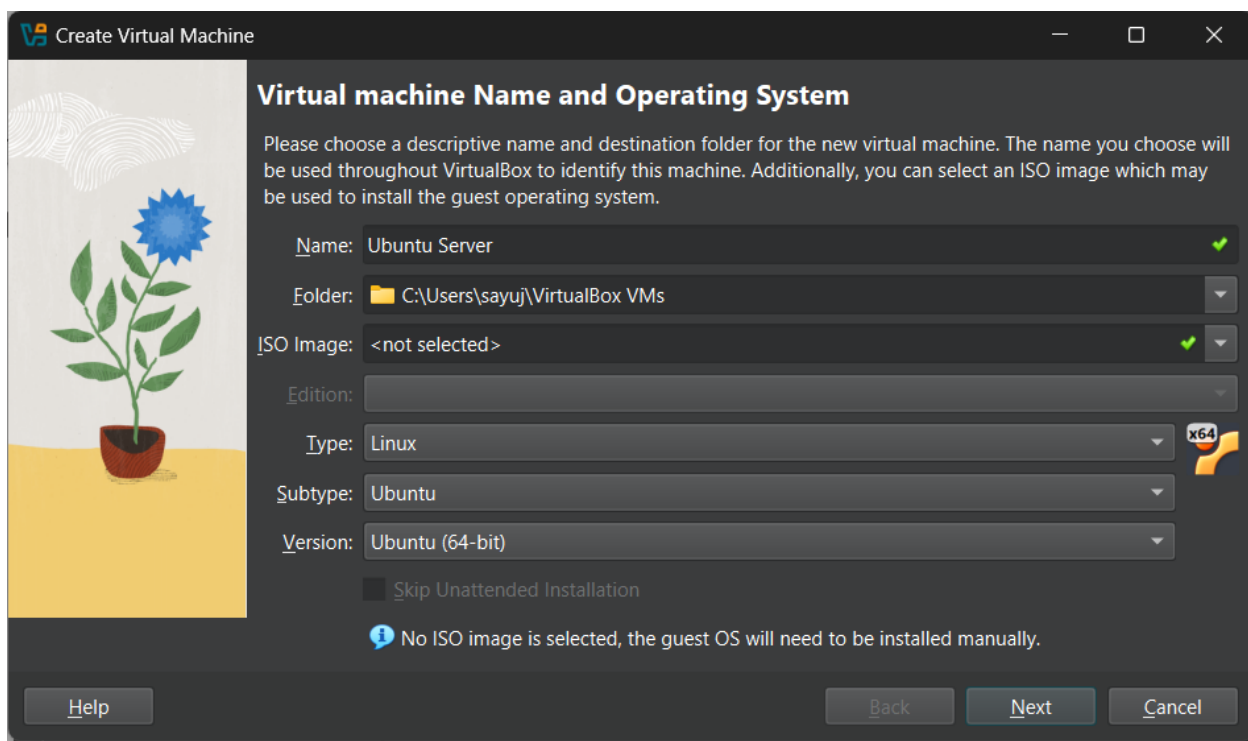


Figure 3.3 Configuring the VM name

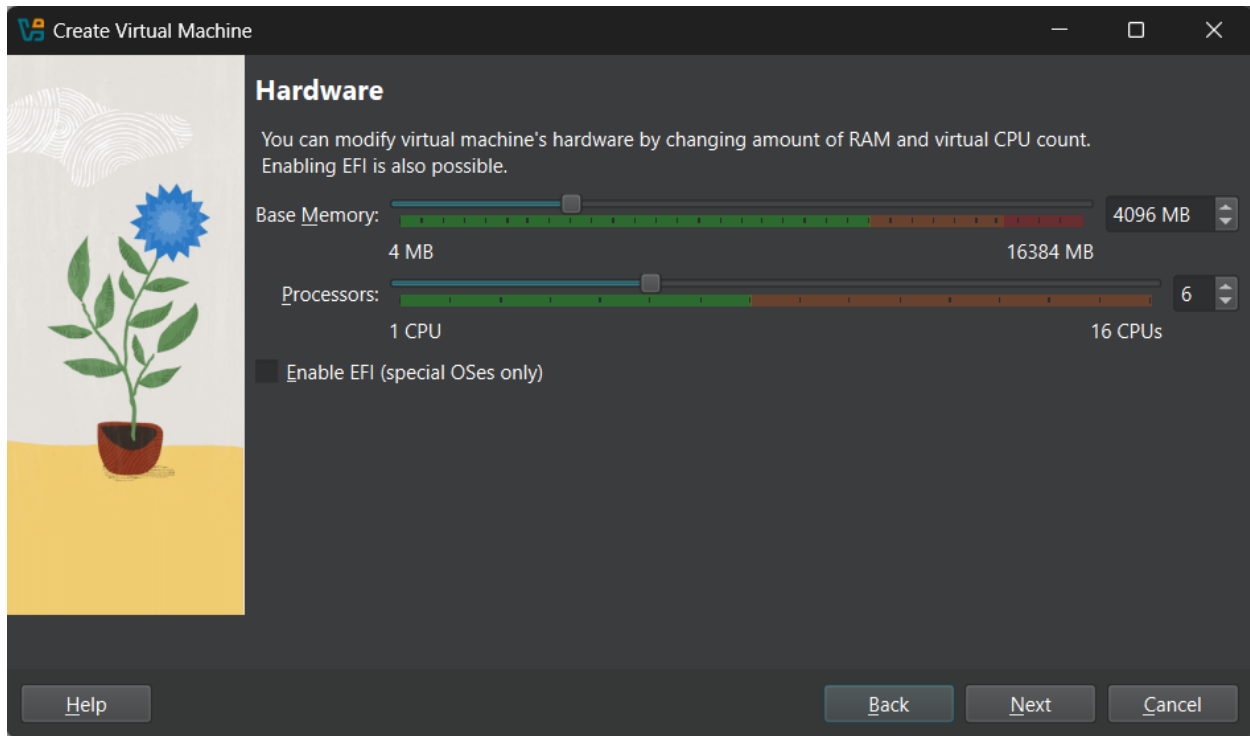


Figure 3.4 Allocating Memory and CPU

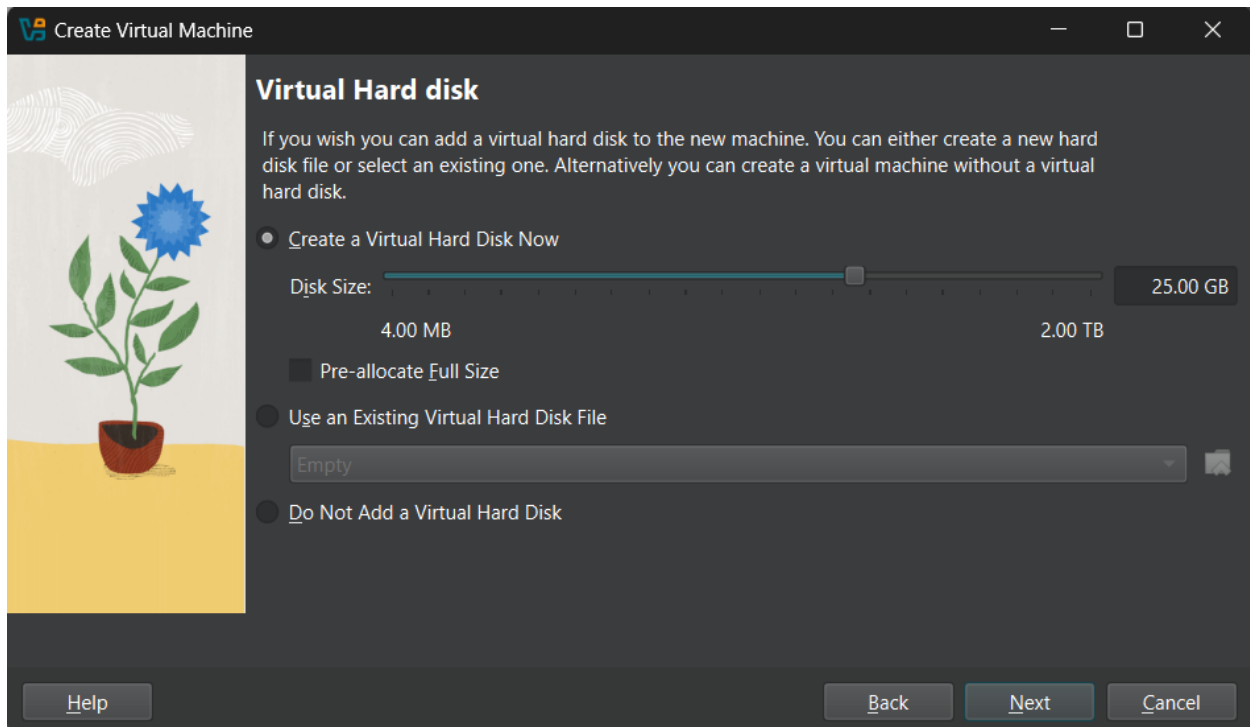


Figure 3.5 Allocating Disk Size

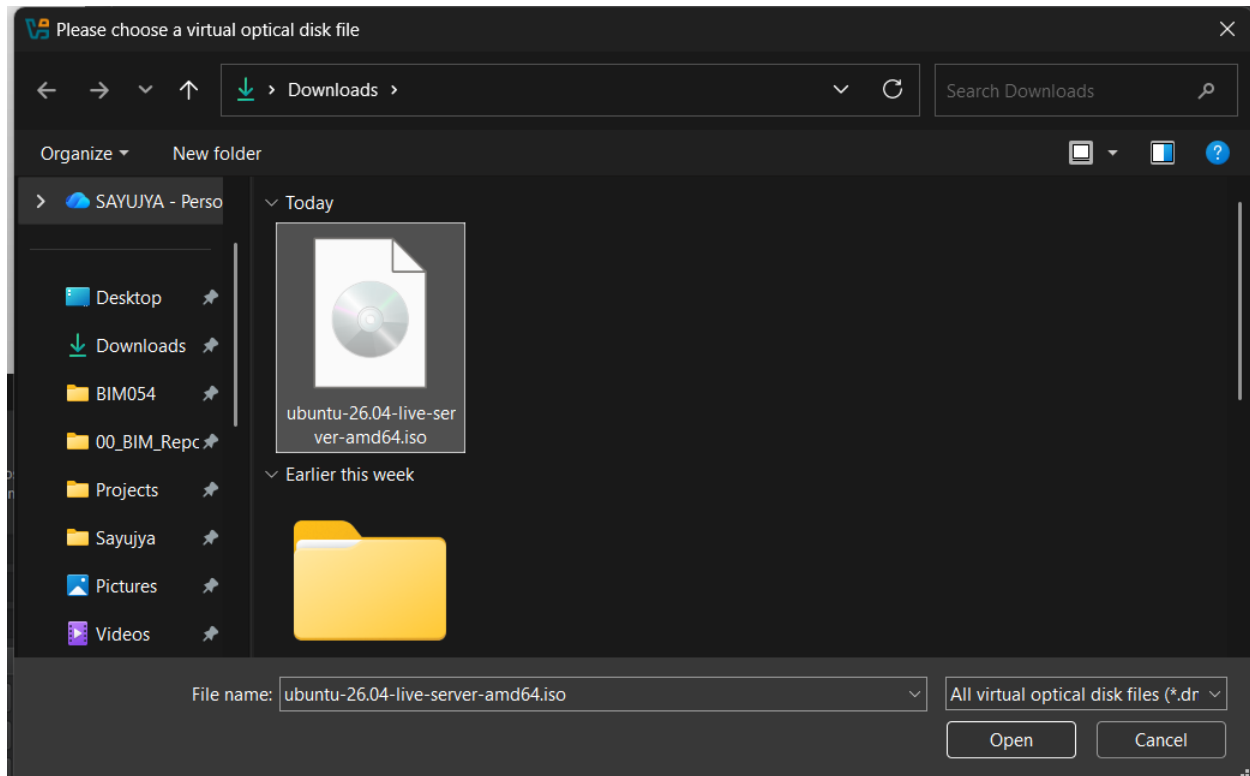


Figure 3.6 Attaching the ubuntu iso

Step 3: Install Ubuntu Server

- Start the virtual machine and follow the Ubuntu Server installation process.
- **Configure:**
 - Username
 - Password
 - Hostname
- Install OpenSSH Server when prompted.

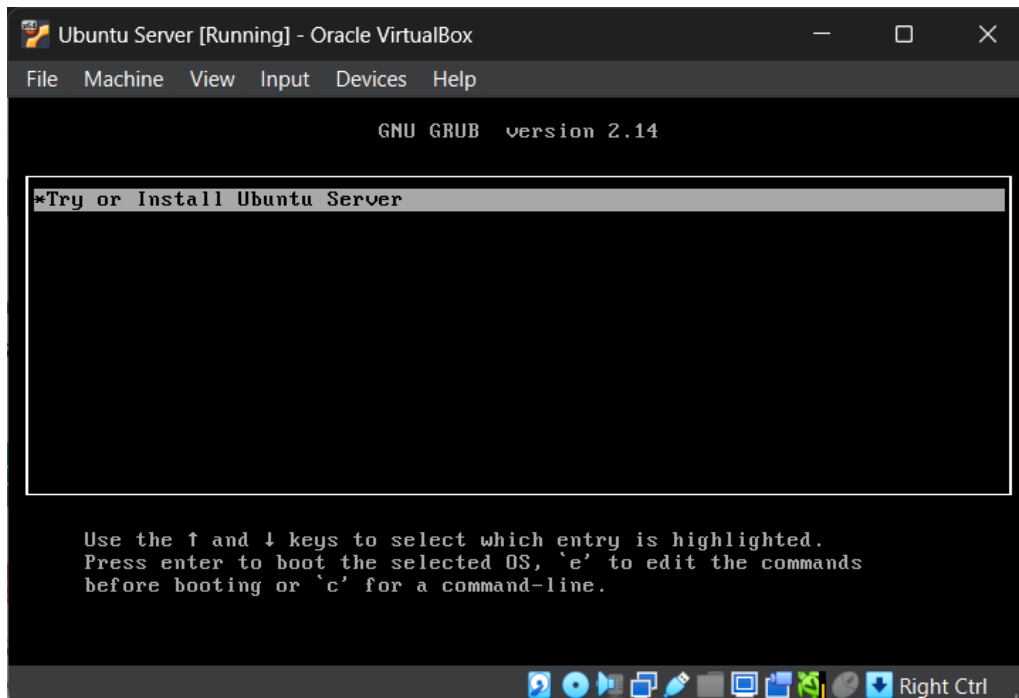


Figure 3.7 Installing Ubuntu Server

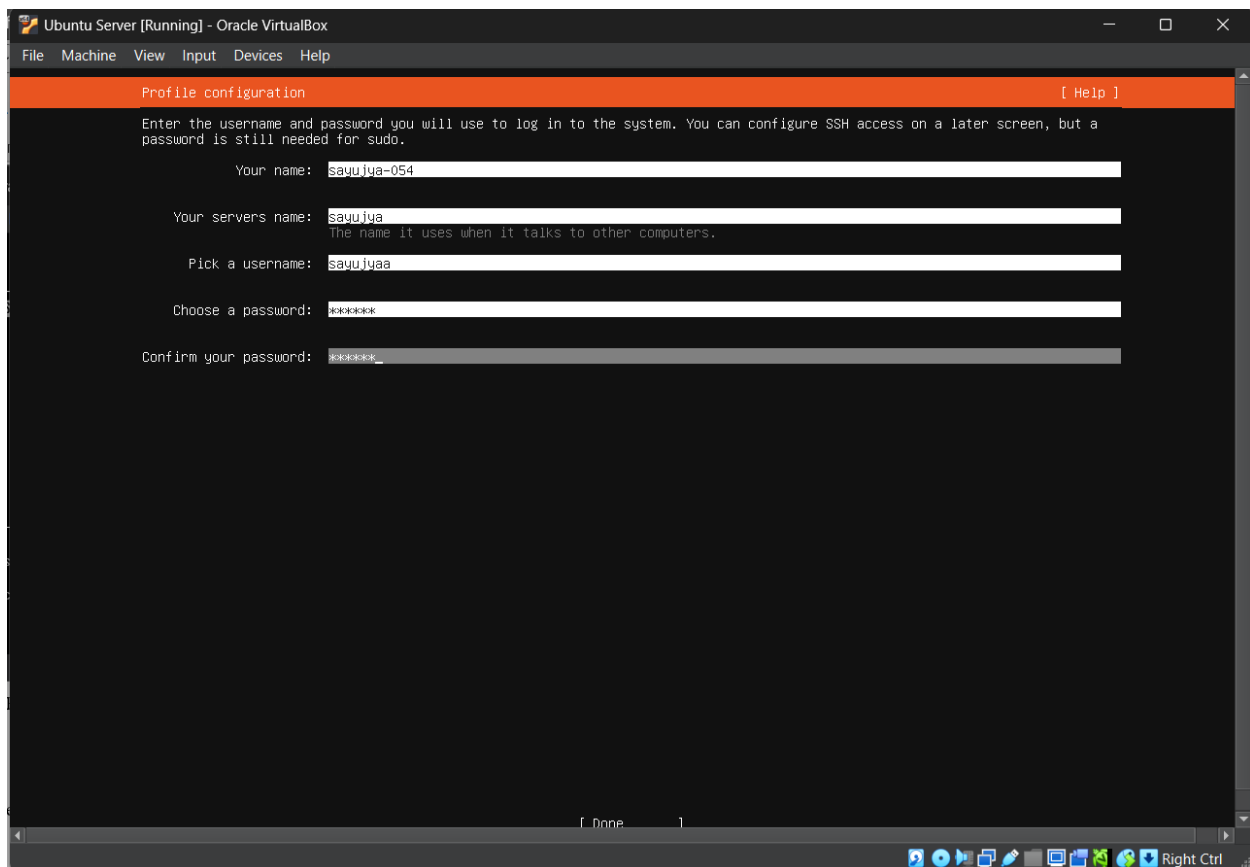
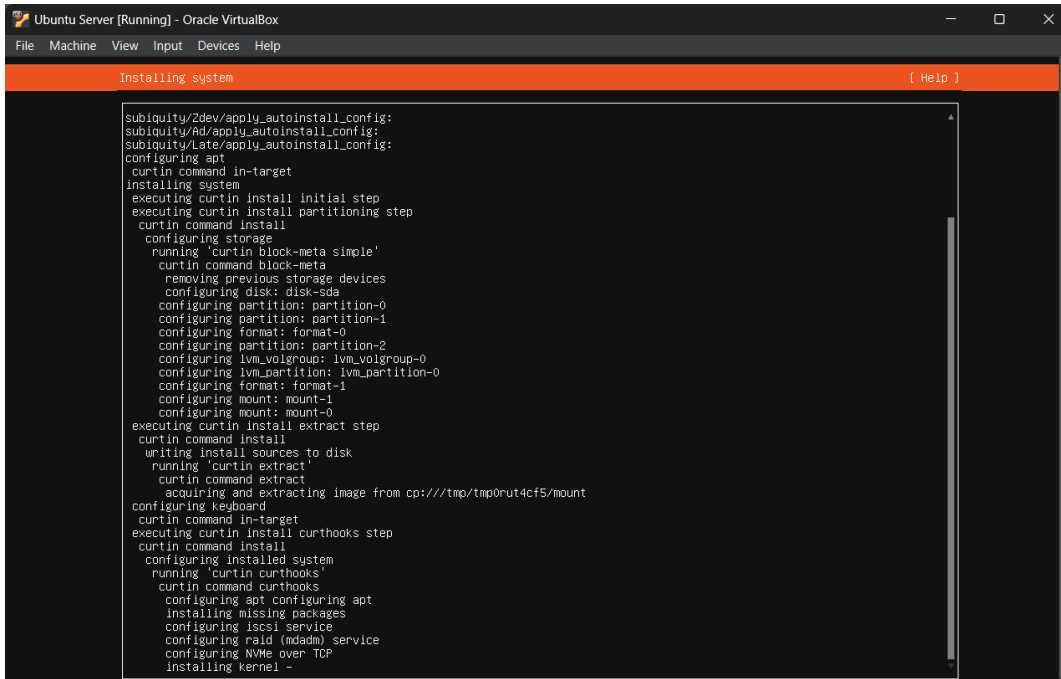


Figure 3.8 Configuration of profile in ubuntu

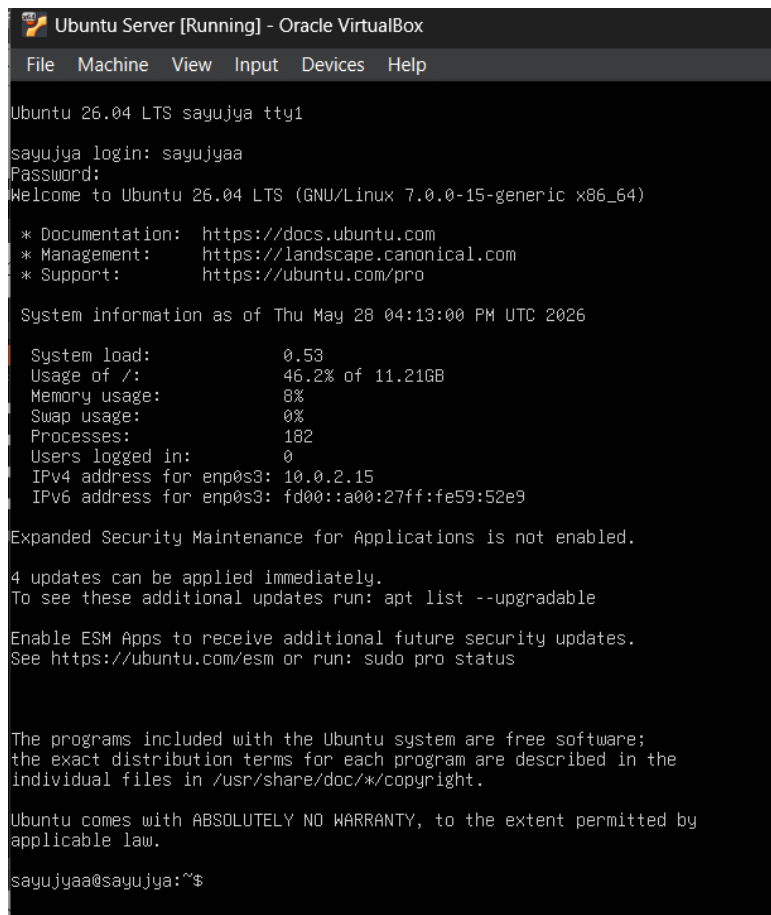


```

subiquity/2dev/apply_autoinstall_config:
subiquity/Ad/apply_autoinstall_config:
subiquity/Late/apply_autoinstall_config:
configuring apt
curtin command in-target
installing system
executing curtin install initial step
executing curtin install partitioning step
curtin command install
configuring storage
running 'curtin block-meta simple'
curtin command block-meta
removing previous storage devices
configuring disk: disk-sda
configuring partition: partition-0
configuring partition: partition-1
configuring format: format-0
configuring partition: partition-2
configuring lvm_voigroup: lvm_voigroup-0
configuring lvm_partition: lvm_partition-0
configuring format: format-1
configuring mount: mount-1
configuring mount: mount-0
executing curtin install extract step
curtin command install
writing install sources to disk
running 'curtin extract'
curtin command extract
acquiring and extracting image from cp:///tmp0rnt4cf5/mount
configuring keyboard
curtin command in-target
executing curtin install curthooks step
curtin command install
configuring installed system
running 'curtin curthooks'
curtin command curthooks
configuring apt configuring apt
installing missing packages
configuring iscsi service
configuring raid (mdadm) service
configuring NVMe over TCP
installing kernel -

```

Figure 3.9 Installation Process



```

Ubuntu 26.04 LTS sayujya tty1
sayujya login: sayujyaa
Password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-15-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Thu May 28 04:13:00 PM UTC 2026

System load:          0.53
Usage of /:            46.2% of 11.21GB
Memory usage:         8%
Swap usage:           0%
Processes:            182
Users logged in:      0
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd00::a00:27ff:fe59:52e9

Expanded Security Maintenance for Applications is not enabled.

4 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

sayujyaa@sayujya:~$

```

Figure 3.10 Successful login to the system

Part 3: Configure NAT Networking and Static IP

Step 1: Configure Network Adapter

Open VirtualBox Settings and then to the Network and set the Adapter 1 to NAT, and Adapter 2 to Host-only Adapter

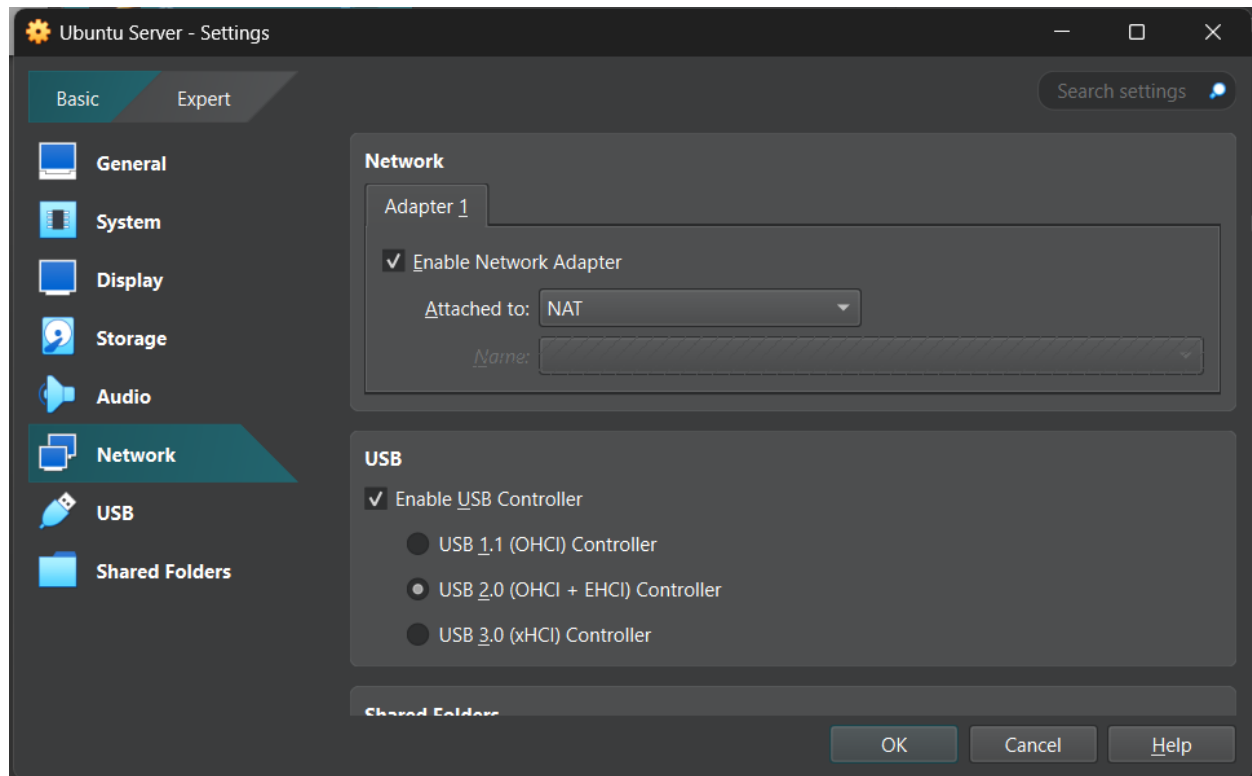


Figure 3.11 Setting network adapter 1 to NAT

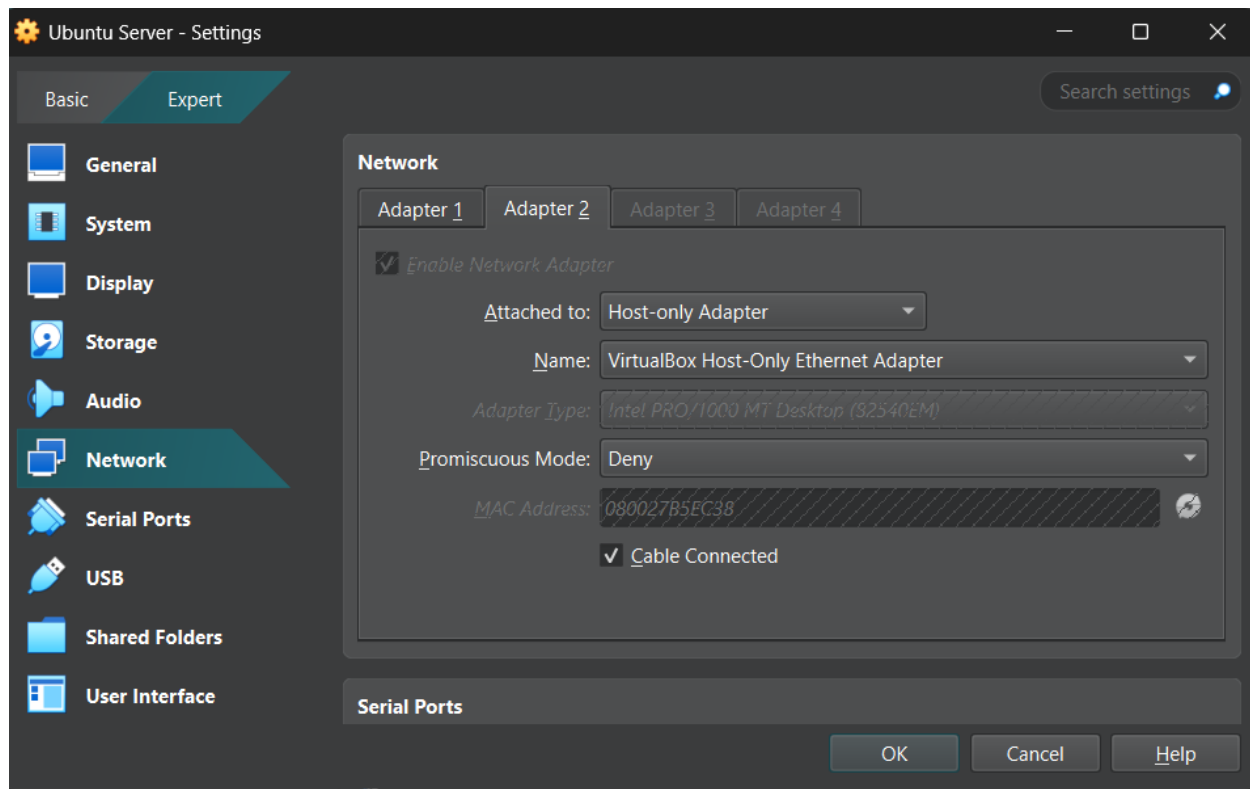


Figure 3.12 Setting Network adapter 2 to Host-only

Step 2: Configure Static IP Address

- First check the IPv4 address from the VMware Tools > Network and inside the VirtualBox Host-only Ethernet Adapter you will see IPv4 Prefix, my is 192.168.56.1/24

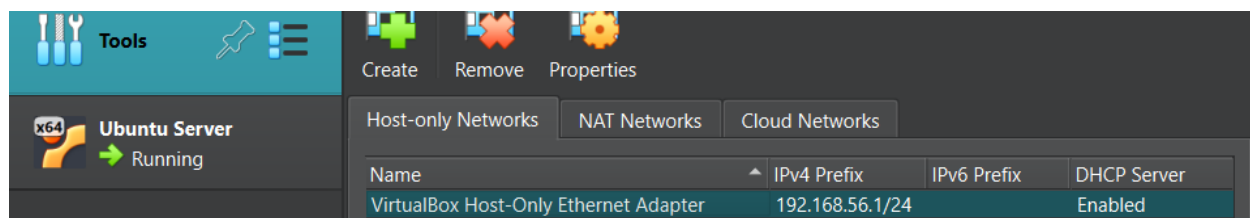


Figure 3.13 IPv4 address of VirtualBox

- Open Netplan configuration file: `sudo nano /etc/netplan/00-installer-config.yaml`
- Set the configurarion to:

```
network:
version: 2
ethernets:
  enp0s3:
    dhcp4: yes
```

```

enp0s8:
  dhcp4: no
  addresses:
    - 192.168.56.100/24

```

- **Apply configuration:** `sudo netplan apply`
- **Verify IP address:** `ip a`

```

network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: yes
    enp0s8:
      dhcp4: no
      addresses:
        - 192.168.56.100/24

```

Figure 3.14 Configuration of the static IP

```

sayujyaa@sayujyaa:~$ sudo netplan apply
sayujyaa@sayujyaa:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:59:52:e9 brd ff:ff:ff:ff:ff:ff
    altname enx0800275952e9
    inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 86216sec preferred_lft 86216sec
    inet6 fd00:a00:27ff:fe59:52e9/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86397sec preferred_lft 14397sec
    inet6 fe80:a00:27ff:fe59:52e9/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:b5:ec:38 brd ff:ff:ff:ff:ff:ff
    altname enx080027b5ec38
    inet 192.168.56.100/24 brd 192.168.56.255 scope global enp0s8
        valid_lft forever preferred_lft forever
    inet6 fe80:a00:27ff:feb5:ec38/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
sayujyaa@sayujyaa:~$

```

Figure 3.15 Verifying the IP address

Part 4: Configure SSH Access

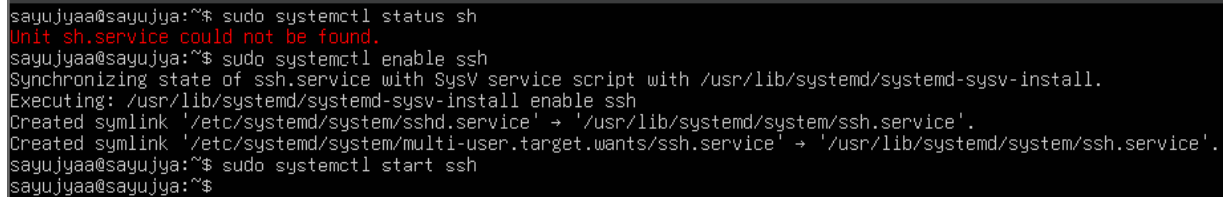
Step 1: Verify SSH Service

- **Check SSH status:** `sudo systemctl status ssh`

- **Enable SSH service:**

```
sudo systemctl enable ssh
```

```
sudo systemctl start ssh
```



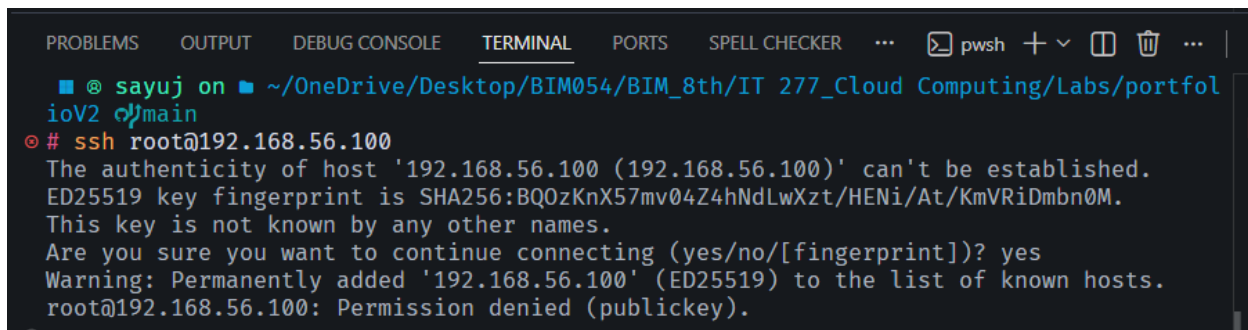
```
sayujyaa@sayujya:~$ sudo systemctl status sh
Unit sh.service could not be found.
sayujyaa@sayujya:~$ sudo systemctl enable ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
Created symlink '/etc/systemd/system/ssh.service' → '/usr/lib/systemd/system/ssh.service'.
Created symlink '/etc/systemd/system/multi-user.target.wants/ssh.service' → '/usr/lib/systemd/system/ssh.service'.
sayujyaa@sayujya:~$ sudo systemctl start ssh
sayujyaa@sayujya:~$
```

Figure 3.16 SSH configuration enable

Step 2: Connect Using VS Code

Open VS Code terminal and run: `ssh root@127.0.0.1 -p 2222`

Enter the server password to connect successfully.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER ... pwsh + v [ ] [ ] ...
■ @ sayuj on ■ ~/OneDrive/Desktop/BIM054/BIM_8th/IT 277_Cloud Computing/Labs/portfol
ioV2 o/main
○ # ssh root@192.168.56.100
The authenticity of host '192.168.56.100 (192.168.56.100)' can't be established.
ED25519 key fingerprint is SHA256:BQOzKnX57mv04Z4hNdLwXzt/HENi/At/KmVRiDmbn0M.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.100' (ED25519) to the list of known hosts.
root@192.168.56.100: Permission denied (publickey).
```

Figure 3.17 Connect to ssh using VS Code

Part 5: Install aaPanel and Nginx

Step 1: Update Ubuntu Server

Run: `sudo apt update && sudo apt upgrade -y`

Step 2: Install aaPanel

- Run the official aaPanel installation command:

```
wget -O install.sh http://www.aapanel.com/script/install_7.0_en.sh
```

```
sudo bash install.sh
```

```
sudo bash install.sh
```

- Wait for installation to complete.
- The installation process generates:
 - aaPanel URL
 - Username
 - Password

```
sayujyaa@sayujya:~$ wget -O install.sh https://www.aapanel.com/script/install_7.0_en.sh
--2026-05-28 17:25:07-- https://www.aapanel.com/script/install_7.0_en.sh
Resolving www.aapanel.com (www.aapanel.com)... 104.26.11.85, 104.26.10.85, 172.67.73.28, ...
Connecting to www.aapanel.com (www.aapanel.com)|104.26.11.85|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 88404 (86K) [application/octet-stream]
Saving to: 'install.sh'

install.sh                               100%[=====]
2026-05-28 17:25:07 (1006 KB/s) - 'install.sh' saved [88404/88404]

sayujyaa@sayujya:~$ sudo bash install.sh

+-----+
| aaPanel FOR CentOS/Ubuntu/Debian |
+-----+
| Copyright © 2015-2099 BT-SOFT(https://www.aapanel.com) All rights reserved. |
+-----+
| The WebPanel URL will be https://SERVER_IP:28634 when installed. |
+-----+

Do you want to install aaPanel to the /www directory now?(y/n):
```

Figure 3.18 Installing aaPanel

```
1
SET_SSL: true
Stopping Bt-Tasks... done
Stopping Bt-Panel... done
Starting Bt-Panel.... done
Starting Bt-Tasks... done

=====
Congratulations! Installed successfully!
=====
aaPanel Internet Address: https://103.10.28.134:28634/51d624f4
aaPanel Internal Address: https://10.0.2.15:28634/51d624f4
username: yc8tp8vj
password: 57263717
Warning:
If you cannot access the panel,
release the following port (28634|888|80|443|20|21) in the security group
=====
Time consumed: 5 Minute!
sayujyaa@sayujya:~$ _
```

Figure 3.19 Complete installation of the aaPanel

Step 3: Install Nginx

- Run: `sudo apt install nginx -y`

```

sayujyaa@sayujyaa:~$ sudo apt install nginx -y
[sudo: authenticate] Password:
The following packages were automatically installed and are no longer required:
  linux-headers-7.0.0-14      linux-image-unsigned-7.0.0-14-generic  linux-modules-7.0.0-14-generic  linux-tools-7.0.0-14
  linux-headers-7.0.0-14-generic  linux-main-modules-zfs-7.0.0-14-generic  linux-tools-7.0.0-14
Use 'sudo apt autoremove' to remove them.

Installing:
  nginx

Installing dependencies:
  nginx-common

Suggested packages:
  fcgiwrap  nginx-doc  ssl-cert

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 0
  Download size: 653 kB
  Space needed: 1,853 kB / 3,734 MB available

Get:1 http://np.archive.ubuntu.com/ubuntu/resolute-updates/main amd64 nginx-common all 1.28.3-2ubuntu1.1 [36.5 kB]
Get:2 http://np.archive.ubuntu.com/ubuntu/resolute-updates/main amd64 nginx amd64 1.28.3-2ubuntu1.1 [616 kB]
Fetched 653 kB in 2s (307 kB/s)
Preconfiguring packages ...
Selecting previously unselected package nginx-common.
(Reading database... 159416 files and directories currently installed.)
Preparing to unpack .../nginx-common_1.28.3-2ubuntu1.1_all.deb...
Unpacking nginx-common (1.28.3-2ubuntu1.1)...
Selecting previously unselected package nginx.
Preparing to unpack .../nginx_1.28.3-2ubuntu1.1_amd64.deb...
Unpacking nginx (1.28.3-2ubuntu1.1)...
Setting up nginx-common (1.28.3-2ubuntu1.1)...
Created symlink '/etc/systemd/system/multi-user.target.wants/nginx.service' → '/usr/lib/systemd/system/nginx.service'.
Setting up nginx (1.28.3-2ubuntu1.1)...
* Upgrading binary nginx

Progress: [ 78%]

```

Figure 3.20 Installing Nginx

Part 6: Configure Firewall

Step 1: Allow Required Ports

- Run the following commands:


```

sudo ufw allow 22
sudo ufw allow 80
sudo ufw allow 8888

```
- Enable firewall: `sudo ufw enable`
- Check firewall status: `sudo ufw status`

Explanation:

- Port 22 → SSH
- Port 80 → HTTP
- Port 8888 → aaPanel Dashboard

```
sayujyaa@sayujya:~$ sudo ufw allow 22
Rule added
Rule added (v6)
sayujyaa@sayujya:~$ sudo ufw allow 80
Rule added
Rule added (v6)
sayujyaa@sayujya:~$ sudo ufw allow 8888
Rule added
Rule added (v6)
sayujyaa@sayujya:~$ sudo ufw enable
Firewall is active and enabled on system startup
sayujyaa@sayujya:~$ sudo ufw status
Status: active
```

To	Action	From
--	-----	----
22/tcp	ALLOW	Anywhere
20/tcp	ALLOW	Anywhere
21/tcp	ALLOW	Anywhere
80/tcp	ALLOW	Anywhere
443/tcp	ALLOW	Anywhere
888/tcp	ALLOW	Anywhere
39000:40000/tcp	ALLOW	Anywhere
28634/tcp	ALLOW	Anywhere
22	ALLOW	Anywhere
80	ALLOW	Anywhere
8888	ALLOW	Anywhere
22/tcp (v6)	ALLOW	Anywhere (v6)
20/tcp (v6)	ALLOW	Anywhere (v6)
21/tcp (v6)	ALLOW	Anywhere (v6)
80/tcp (v6)	ALLOW	Anywhere (v6)
443/tcp (v6)	ALLOW	Anywhere (v6)
888/tcp (v6)	ALLOW	Anywhere (v6)
39000:40000/tcp (v6)	ALLOW	Anywhere (v6)
28634/tcp (v6)	ALLOW	Anywhere (v6)
22 (v6)	ALLOW	Anywhere (v6)
80 (v6)	ALLOW	Anywhere (v6)
8888 (v6)	ALLOW	Anywhere (v6)

```
sayujyaa@sayujya:~$
```

Figure 3.21 Setup firewall

Part 7: Access aaPanel Dashboard

- Open browser and visit: `http://192.168.56.100:28634/51d624f4` (The IP is the same one used in the ssh one)
- Login using the credentials generated during installation.

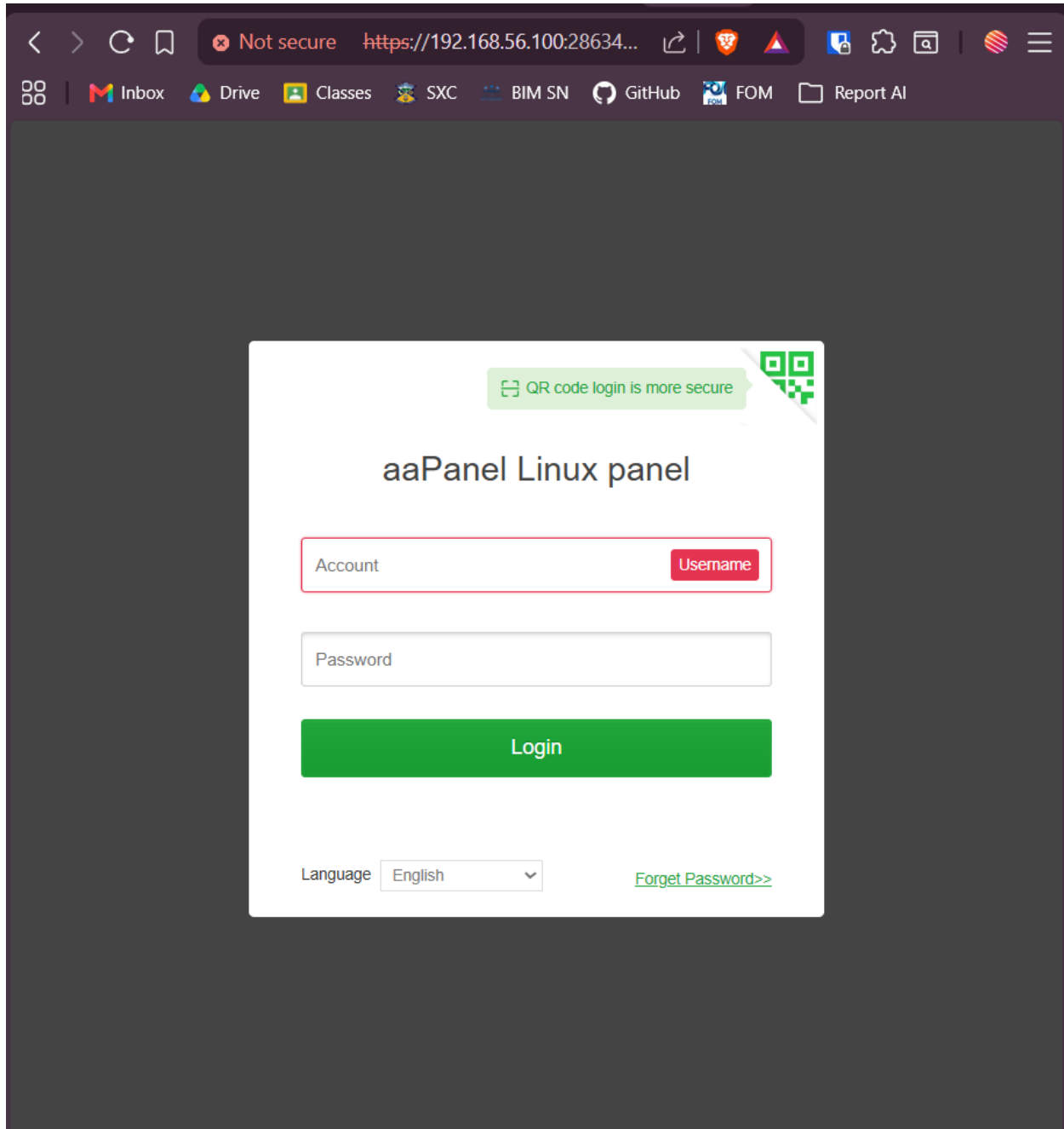


Figure 3.22 The aaPanel login

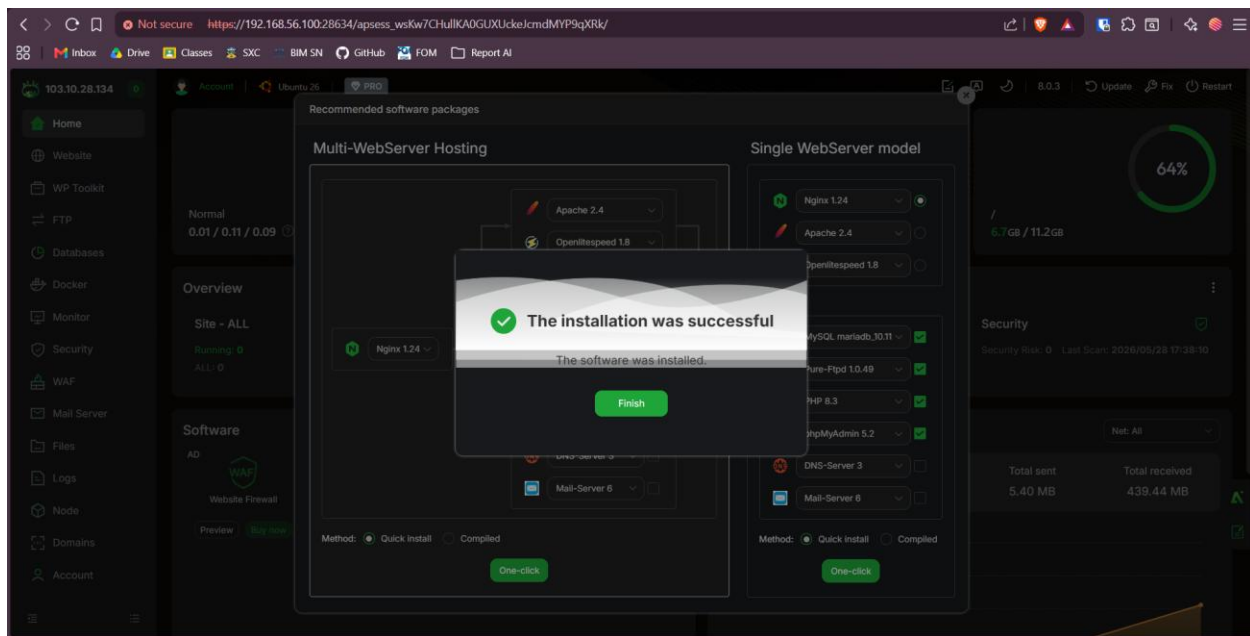


Figure 3.23 Successful aaPanel login

Part 8: Verify Nginx Deployment

- Open browser and visit: `http://192.168.56.100`
- The default Nginx welcome page should appear.

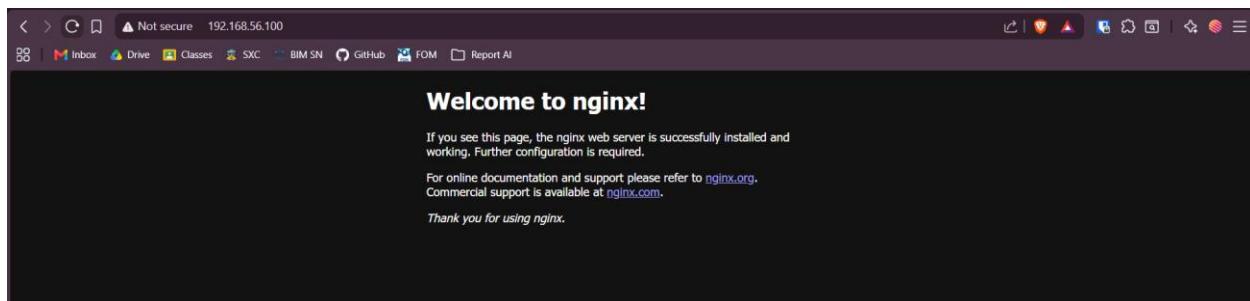


Figure 3.24 Default Nginx page

Result

Oracle VM VirtualBox and Ubuntu Server 26.04 were successfully installed. NAT networking and static IP configuration were completed successfully. SSH access was established using VS Code terminal. aaPanel and Nginx were installed and configured successfully, and the dashboard was accessible through the browser.

Conclusion

This lab provided practical experience in server virtualization, Linux server administration, SSH remote access, firewall management, and web hosting panel deployment. The experiment demonstrated how cloud and server infrastructure can be managed using virtualization technologies and Linux-based hosting environments.